

2026 年 5 月 21 日 Version 1.0

ハッカソン ～計画書と設計書～

チーム 23

24G1038 : 片岡 聖

24G1175 : 地曳 賢人

24G1131 : 御代川 稜

25G1021 : 梅澤 颯太

25G1053 : 齋藤 翔太

25G1065 : 塩澤 匠生

目次

第 1 章	プロジェクトの計画書	1
1.1	プロジェクトの概要	1
1.1.1	プロジェクトの題目	1
1.1.2	プロジェクトの目的	1
1.1.3	既存の技術とプロジェクトの独自性	2
1.2	スコープ・マネジメント	2
1.2.1	成果物	2
1.2.2	プロジェクトの対象範囲と非対象範囲	3
1.2.3	機能分解構造 FBS	3
1.2.4	製品分解構造 PBS	4
1.2.5	FBS と PBS の対応関係	4
1.3	作業計画	5
1.3.1	作業分解構造 WBS と管理番号	5
1.3.2	アローダイアグラムとクリティカルパス	6
1.3.3	役割分担	7
1.3.4	ガントチャート	7
1.4	検証と妥当性確認: V&V 計画	8
1.4.1	プロジェクトの成果物に対する有効指標 MOE	8
1.4.2	有効指標 MOE に対応する性能指標 MOP	9
1.4.3	システムテストで評価する性能指標 MOP	9
1.4.4	結合テストで評価する性能指標 MOP	10
1.4.5	単体テストで評価する性能指標 MOP	10
1.4.6	プロジェクト中に監視する技術性能指標 TPM	10
1.4.7	プロジェクト中に監視する指標 TPM	10
1.5	資源・調達管理	11
1.6	リスク管理	11
第 2 章	システムの基本設計	13

2.1	機能一覧	13
2.1.1	要求一覧	13
2.1.2	機能分解	14
2.2	システム構成図	15
2.2.1	全体構成	15
2.2.2	ノードとパートの対応	16
2.2.3	ノードの役割分担	16
2.2.4	演奏中のデータフロー	17
2.2.5	採用アーキテクチャ (EMA)	17
2.3	操作インターフェース設計	18
2.3.1	ユーザー操作	18
2.3.2	LED 表示仕様	19
2.3.3	通信メッセージ	19
2.4	状態遷移図	20
2.4.1	指揮者ノード (node_01)	20
2.4.2	楽器ノード (node_02~06)	21
2.4.3	PC (Processing)	21
第 3 章	システムの詳細設計	23
3.1	部品一覧	23
3.2	ハードウェア設計	24
3.2.1	配線・ピンアサイン	25
3.2.2	ネットワーク構成	25
3.3	ソフトウェア設計	26
3.3.1	ファイル構成	26
3.3.2	オブジェクト設計 (クラス図)	27
3.3.3	通信パケット設計	28
3.3.4	楽譜データ形式	29
3.3.5	時刻同期方式	30
3.3.6	音色合成 (PC 側)	31
3.4	処理フローの設計	31
3.4.1	3 フェーズ実行モデル	32
3.4.2	指揮者ノードの処理フロー	32

3.4.3	楽器ノードの処理フロー	33
3.5	テストの設計	34
3.5.1	設計目標値	35
3.5.2	検証の方向性	35
3.5.3	実機検証で確定する仮値	36
第 4 章	生成 AI の利用	39

第 1 章 プロジェクトの計画書

本章では、本プロジェクトの概要・スコープ・作業計画・検証方針・資源・リスクを示す。本プロジェクトは約 13 週間をかけ、ハードウェア（Arduino）とソフトウェア（Processing）を組み合わせた電子オーケストラを構築するものである。チーム 23 では、指揮動作によって楽曲のテンポと音量を変化させられる演奏システムを開発する。

1.1 プロジェクトの概要

1.1.1 プロジェクトの題目

本プロジェクトの題目は「指揮用 Arduino から指揮をするオーケストラ」である。指揮者が手に持つ Arduino の動きを起点として、複数の楽器ノードと PC アプリが同期して合奏する、体験型の電子オーケストラを指す。

1.1.2 プロジェクトの目的

本プロジェクトの目的は、時間帯・天候・通信環境などの外部条件や明るさの制約に左右されず、音楽経験のない人でも直感的な指揮動作だけで演奏に参加できるシステムを開発することである。具体的には、指揮棒を振る周期を検出して楽曲のテンポへ反映し、加速度の大きさに応じた音量変化や LED による視覚的演出を加えることで、誰でも楽しめる体験型電子オーケストラの実現を目指す。最終的なゴールは、指揮棒を振った際に、振りの大きさや周期の速さに応じて音楽の音量とテンポが変化する状態を達成することである。

本目的の達成には、次の課題を解決する必要がある。

- ・ **指揮動作の安定検出**：指揮棒の振りには個人差があり、速さ・大きさ・角度が異なる。どのような振り方でも安定して動作を検出する必要がある。

- ・ **テンポへのリアルタイム反映**：指揮棒を振った周期に合わせ、遅延なく自然にテンポを変化させる必要がある。反応が遅いと操作感が損なわれる。
- ・ **誤検出の抑制**：小さな揺れや不要な動きを指揮動作と誤認するとテンポが乱れるため、必要な動作だけを判定する仕組みが要る。
- ・ **未経験者でも使える操作性**：専門知識がなくても直感的に扱えるよう、簡単な操作方法と分かりやすいフィードバックを用意する。
- ・ **演奏状態の可視化**：現在のテンポや音量変化が伝わりにくいと操作しづらいため、LED 表示などで演奏状態を可視化する。
- ・ **環境条件への非依存**：明るさ・時間帯・天候・通信環境に依存せず、屋内外で安定して使用できる構成とする。

1.1.3 既存の技術とプロジェクトの独自性

既存技術として、テルミンのような非接触型電子楽器や、カメラ・センサを用いて身体動作を音楽表現へ変換するシステムが存在する [1]。また、Wii Music[2] では、コントローラの動作から拍を検出して音楽のテンポを制御している。しかし、これらの多くは高い演奏技術を要したり、明暗などの環境条件に影響を受けやすい。さらに、1 台の機器内で音源生成を完結させる構成が多く、複数端末による分散演奏や、身体動作によるリアルタイムな指揮制御には十分対応していない。以上より、複数マイコンによる分散演奏と、身体動作によるリアルタイム指揮制御を両立する点に、本プロジェクトの新規性がある。

1.2 スコープ・マネジメント

1.2.1 成果物

本プロジェクトの成果物を以下に示す。

- ・ オーケストラ実機一式（指揮者ノード 1 台＋楽器ノード 5 台）
- ・ 金管とドラムの音色を合成する Processing プログラム（各楽器ノードに 1 対 1 で接続する 5 台の PC 上で動作）
- ・ システム構成図および設計書一式
- ・ 動作確認結果および性能評価資料

- ・プロジェクト計画書および最終報告書

1.2.2 プロジェクトの対象範囲と非対象範囲

本プロジェクトの対象範囲を以下に示す。

- ・ Arduino を用いた指揮動作検出システムの開発
- ・ 加速度センサによる指揮棒の振り動作の取得
- ・ 振り周期に応じた楽曲テンポの制御機能
- ・ 振りのエッジ検出によるテンポ変化機能
- ・ 振り速度に応じた音量変化機能
- ・ ハードウェアとソフトウェアを統合したシステム構築
- ・ システムの動作確認および性能評価

一方、以下の内容は対象外とする。

- ・ プロのオーケストラに匹敵する高精度な指揮解析
- ・ AI を用いた自動指揮認識機能
- ・ 複数人による同時指揮への対応
- ・ 市販音楽制作ソフトウェアとの連携
- ・ 楽曲の自動生成機能、および複雑なジェスチャ認識機能

1.2.3 機能分解構造 FBS

本システムが実現すべき機能を階層的に整理した機能分解構造（FBS）を図 1.1 に示す。最上位の「Arduino オーケストラ」を、指揮解析・同期通信・楽譜進行・音響合成の 4 つの機能群へ分解した。指揮解析は指揮動作の認識から演奏速度の決定・開始停止・表情抽出までを担い、同期通信は共通時刻の共有・指示の一斉配信・通信の安定性確保を担う。楽譜進行はテンポ追従再生・パート別演奏制御・発音タイミング管理を、音響合成は金管楽器音の再現・多重音の発音・演奏状態の可視化を担う。

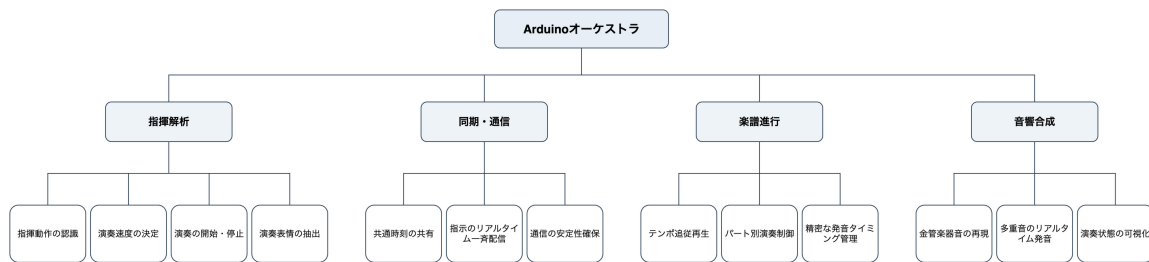


図 1.1 機能分解構造 (FBS)

1.2.4 製品分解構造 PBS

FBS の各機能を実現するために必要な成果物を整理した製品分解構造 (PBS) を図 1.2 に示す。システムを、指揮者ノードユニット・ネットワーク基盤・楽器演奏ユニット・音響合成アプリケーションの 4 系統へ分解した。指揮者ノードユニットは指揮者用 Arduino と動作解析ファームウェア・コマンド送出モジュールから成り、ネットワーク基盤は全ノードで共有する通信モジュール・共通定義ファイル・API フレームワークから成る。楽器演奏ユニットは楽器用 Arduino と演奏進行エンジン・楽譜データアセットを、音響合成アプリケーションは Processing 音響合成エンジン・ボイスマネージャー・演奏モニタ UI を含む。

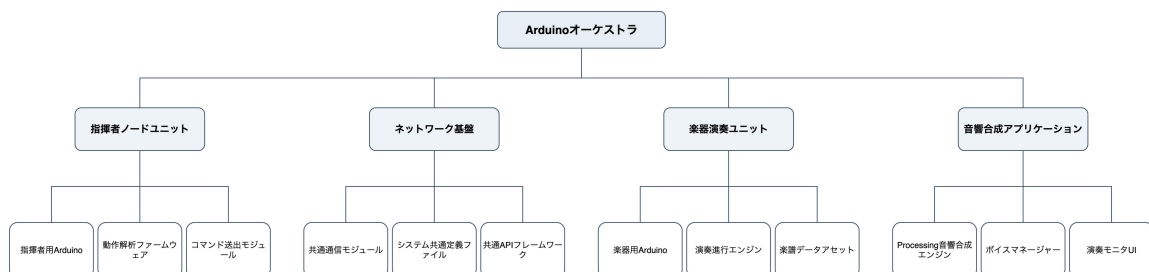


図 1.2 製品分解構造 (PBS)

1.2.5 FBS と PBS の対応関係

FBS はシステムが実現する「機能」を、PBS はそれを実現する「成果物・構成要素」を表す。両者の主な対応関係を表 1.1 に示す。FBS で定義した各機能群は、PBS のいずれかの成果物系統によって実現される構成となっている。

表 1.1 FBS と PBS の対応関係

FBS（機能群）	対応する PBS（成果物系統）
指揮解析	指揮者ノードユニット（指揮者用 Arduino, 動作解析ファームウェア, コマンド送出モジュール）
同期・通信	ネットワーク基盤（共通通信モジュール, システム共通定義ファイル, 共通 API フレームワーク）
楽譜進行	楽器演奏ユニット（楽器用 Arduino, 演奏進行エンジン, 楽譜データアセット）
音響合成	音響合成アプリケーション（Processing 音響合成エンジン, ボイスマネージャー, 演奏モニタ UI）

1.3 作業計画

1.3.1 作業分解構造 WBS と管理番号

本プロジェクトの作業を，設計・製造・テスト・ストレッチ・運用発表の 5 つのフェーズに分け，要素成果物ごとに管理番号を付して整理した。作業分解構造（WBS）を表 1.2 に示す。番号は，フェーズ（100 番台～500 番台）と要素成果物（下 2 桁）の体系で付与している。

表 1.2 作業分解構造 (WBS)

番号	要素成果物	主なタスク	担当
100 設計フェーズ (工数目安 3 週間)			
110	基本設計	FBS / PBS / MOE・MOP・TPM の最終化, システムアーキテクチャ・通信プロトコル・楽譜データ形式・音色合成の基本設計	全員, 塩澤, 齋藤, 梅澤
120	詳細設計	共通層 API・指揮者・楽器の詳細設計, Processing 詳細設計, 周波数分析, 楽譜データ詳細設計	塩澤, 梅澤, 齋藤
200 製造フェーズ (工数目安 2 週間)			
210	共通層実装	IModule / ModuleTimer 実装, SystemData / ProjectConfig 整備, OrcNetModule 実装	塩澤
220	指揮者ノード	IMU ドライバ, 信号前処理, 拍検出, テンポ推定, コマンド送出手の各モジュール実装	塩澤
230	楽器ノード	CTRL / BEAT 受信, 楽譜進行ロジック, NOTE 送出手, パート別差分適用の実装	塩澤
240	楽譜準備	課題曲 5 パートの選定, 楽譜ヘッダの配列化, ROM 埋め込み確認	齋藤
250	Processing 実装	NOTE 受信, 音色合成エンジン, 多重発音管理, UI / モード表示	梅澤
300 テストフェーズ (工数目安 2 週間)			
310	単体テスト	共通層・拍検出・楽譜進行・音色生成の単体動作確認	塩澤, 梅澤
320	結合テスト	指揮者単体デモ, 指揮者・楽器結合, 5 ノード同期誤差計測, 全ノード結合	塩澤, 全員
330	システムテスト	拍検出精度, 通信遅延, 同期誤差, テンポ追従, パケットロス耐性, CPU 負荷, 起動時間	塩澤, 全員
400 ストレッチフェーズ (余裕がある場合)			
410	強弱推定	velocity 推定アルゴリズム実装, CTRL パケットへの乗せこみ, 強弱制御テスト	塩澤
500 運用・発表フェーズ			
510	発表	発表準備・デモ調整	全員
520	報告書	成果報告書 (個人) の作成	全員

1.3.2 アローダイアグラムとクリティカルパス

WBS の要素成果物間の先行関係をアローダイアグラムとして図 1.3 に示す。クリティカルパスは「110 基本設計 → 120 詳細設計 → 210 共通層実装 → 220 指揮者ノード → 230 楽器ノード → 310/320 テスト → 330 システムテスト → 510 発表

準備」で、全長は約 8 週間である。240 楽譜準備と 250Processing 実装は、120 詳細設計の完了後にクリティカルパスと並行して進められ、それぞれ 0.5～1.0 週のフロート（余裕）を持つため、多少の遅延を吸収できる。

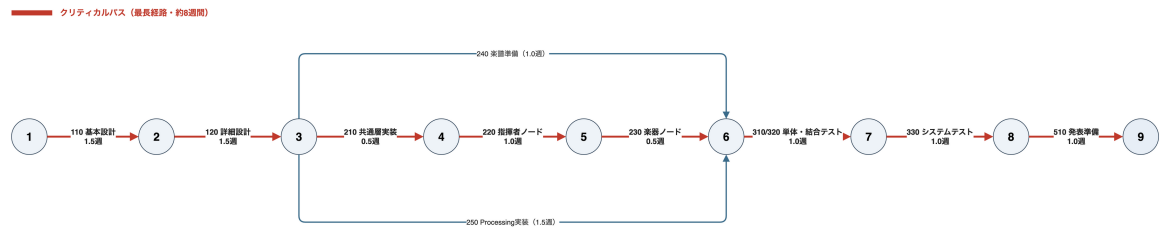


図 1.3 アローダイアグラムとクリティカルパス

1.3.3 役割分担

チーム 23 は 6 名で構成される。役割分担を表 1.3 に示す。Arduino 系プログラム（指揮者・楽器・共通層）は、モジュール境界をそろえる目的で塩澤が一括して設計・実装する。楽譜データは齋藤，Processing による音色合成は梅澤が担当し，御代川・地曳・片岡は議事録・計画書編纂・調達を担う。テストフェーズ以降の実機検証および発表準備には全員が合流する。

表 1.3 チーム 23 の役割分担

メンバー	学籍番号	主な担当
塩澤 匠生	25G1065	Arduino 系プログラム全般（指揮者・楽器・共通層の設計・実装）
齋藤 翔太	25G1053	楽譜データ作成・音階生成ロジック・周波数分析
梅澤 颯太	25G1021	Processing による音色合成・演奏再生・UI
御代川 稜	24G1131	議事録（持ち回り 1 番手），計画書編纂
地曳 賢人	24G1175	議事録（2 番手），スケジュール管理
片岡 聖	24G1038	議事録（3 番手），WBS・役割整理，機材調達

1.3.4 ガントチャート

WBS と授業日程に基づくガントチャートを図 1.4 に示す。設計フェーズ（4 月下旬～5 月）で FBS・PBS・MOE/MOP/TPM の最終化と基本・詳細設計を行い，製造フェーズ（5 月下旬～6 月中旬）で共通層・指揮者・楽器・楽譜・Processing

の実装を並行して進める。テストフェーズ（6 月下旬）で単体・結合・システムテストを段階的に実施し、余裕があればストレッチとして強弱推定を追加する。運用・発表フェーズ（7 月）で成果発表会と成果報告書の作成を行う。240 楽譜準備と 250Processing 実装に余裕時間を持たせ、クリティカルパスの遅延を並列タスクで吸収できる構成とした。

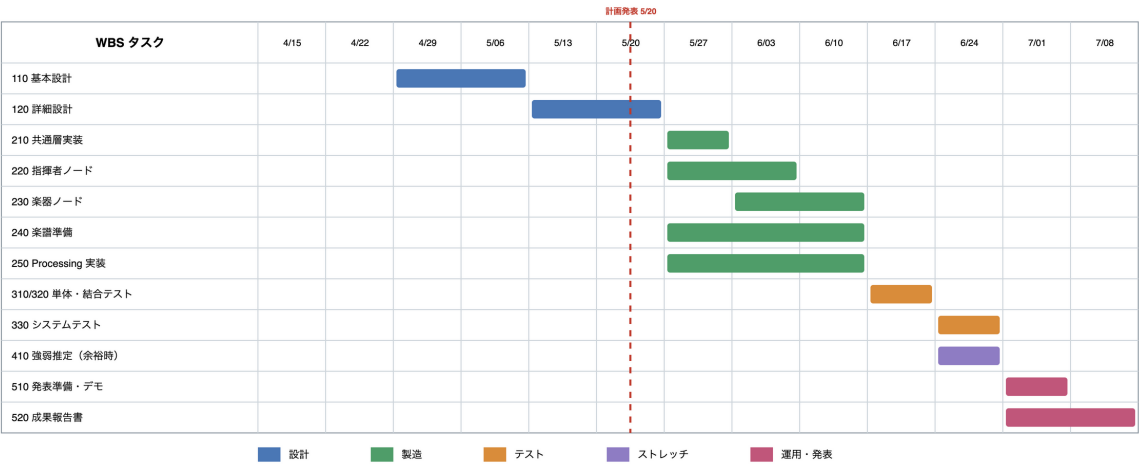


図 1.4 ガントチャート（WBS と授業日程の対応）

1.4 検証と妥当性確認: V&V 計画

1.4.1 プロジェクトの成果物に対する有効指標 MOE

有効指標（MOE：Measure of Effectiveness）は、システムが最終的にユーザー（指揮者・聴衆）へ提供する価値を評価する指標である。本プロジェクトでは 3 つの MOE を定め、各テスト階層の性能指標（MOP）の達成を通じて有効性を確立する。MOE と MOP の対応関係および目標値を表 1.4 に示す。

表 1.4 MOE と MOP の対応関係および目標値

性能指標 (MOP)	目標値
MOE 1：音楽的再現性と正確性（単体テスト領域）	
拍検出の正確性	正解率 $\geq 90\%$
音階の誤差	平均 $\leq 5.52 \text{ cent}$
楽譜との相違	誤ノート発音数 0
MOE 2：合奏の一体感とインタラクティブ性（結合テスト領域）	
楽器間同期誤差	$\leq 20 \text{ ms}$
指揮から楽器への通信遅延	$\leq 30 \text{ ms}$
テンポ追従の遅延	$\leq 2 \text{ 拍}$
MOE 3：システムの堅牢性と実用性（システムテスト領域）	
起動時間	$\leq 5 \text{ s}$
CPU 負荷（入力フェーズ）	$\leq 2 \text{ ms}$
パケットロス耐性	演奏継続が可能なレベル

1.4.2 有効指標 MOE に対応する性能指標 MOP

前項の各 MOE に対し，システム全体として達成すべき性能指標を次の通り定める。これらは特定の検証フェーズに限定されず，成果物が最終的に備えるべき技術的到達基準である。

- ・ **音楽的品質の達成基準**：指揮動作から拍を正確に解析し，楽譜データを正確なピッチで音響へ変換する精度を主軸に評価する。
- ・ **リアルタイム・アンサンブルの達成基準**：分散配置した複数ノード間の極小な同期誤差と，テンポ変化への停滞のない追従性能を主軸に評価する。
- ・ **運用プラットフォームの達成基準**：限られた計算資源内での処理完結，無線環境下での通信断耐性，即時稼働を可能にする運用性を主軸に評価する。

1.4.3 システムテストで評価する性能指標 MOP

システムテストでは，パケットロス耐性と CPU 負荷を評価する。パケットロスは，聴感上の破綻が生じない 5%以内を許容範囲の指標とする。CPU 負荷は，入力フェーズの処理時間を制御周期内に収めるため，2 ms 以内を指標とする。

1.4.4 結合テストで評価する性能指標 MOP

結合テストでは、ノード間の同期と遅延を評価する。20 ms を超えるずれがあると 2 つの音を別々の発音として知覚するため、楽器間の同期誤差は 20 ms 以内を指標とする [3]。指揮棒から楽器への通信遅延は 30 ms 以内、テンポ追従の遅延は 2 拍以内を指標とする。

1.4.5 単体テストで評価する性能指標 MOP

単体テストでは、拍検出と音階解析の精度を評価する。拍検出の正確性は 90%以上を指標とする。音階の誤差は、自己相関による音階解析で平均 5.52 cent であったことを踏まえ、5.52 cent までの誤差を指標とする [6]。

1.4.6 プロジェクト中に監視する技術性能指標 TPM

開発期間中、各性能指標の達成見込みを継続的に評価するための技術性能指標 (TPM) として次を監視し、技術的リスクを早期に発見する。

- ・ **同期誤差の推移**：目標値 20 ms に対する到達度を監視する。通信プロトコルの最適化やノード数の増減が生じた際に実測し、目標値からの乖離が 10 ms 以上となった場合はバッファ制御を再検討する。
- ・ **CPU 処理時間のマージン**：制御周期 5 ms に対する各タスクの処理時間を監視する。CPU 負荷が目標値 (2 ms) の 80%を超えた段階で、コード最適化とタスク優先順位の再設計を行う。

1.4.7 プロジェクト中に監視する指標 TPM

技術的な数値指標に加え、プロジェクト全体の進捗と資源の健全性を維持するため次を監視する。

- ・ **機能実装の完了率**：WBS に基づく各モジュールの実装進捗を監視する。指揮解析と通信基盤の統合が遅延した場合は、クリティカルパス上のリスクとして要員配置やスコープの調整を行う。
- ・ **システム稼働の安定度**：連続稼働テストにおける意図しないリセットやハングアップの発生頻度を監視し、不安定要素やメモリリークを早期に排除する。

1.5 資源・調達管理

本プロジェクトの遂行に必要なハードウェア資源および開発環境の調達計画を次に示す。

- ・ **ハードウェア資源**：楽器ノード用に Arduino UNO R4 WiFi を 5 台，指揮者ノード用に XIAO ESP32-S3 Sense を 1 台使用する。さらに，各楽器ノードに 1 対 1 で接続し音色合成を担う PC を 5 台用いる。指揮解析用の IMU センサおよび各楽器ノードの出力デバイスは，チーム内の既存資産と新規調達によって確保済みである。
- ・ **開発・運用リソース**：ソースコード管理に Git を用い，チーム内の並行開発を可能とする。無線通信の検証用に専用ルーターを用い，会場等の外部環境に依存しない安定した通信経路を確保する。

1.6 リスク管理

各担当箇所の作業日数を長めに見積もり，バッファを設けることで，体調不良や不具合により作業が滞った場合でも，スケジュール通りへの修正を容易にする。特に，クリティカルパス上にない 240 楽譜準備・250Processing 実装に余裕時間を持たせ，クリティカルパスに遅延が生じても並列タスクで吸収できる構成とした。進捗の遅れは前項の TPM（機能実装の完了率）で早期に検知し，要員配置やスコープの調整で対応する。

第 2 章 システムの基本設計

本章では、第 1 章で定めた計画に基づき、Arduino オーケストラの基本設計を示す。システムは**指揮者マイコン 1 台・楽器マイコン 5 台・PC (Processing) 5 台**で構成される。指揮者の振りを起点として、楽器マイコンが楽譜を進行させ、各 PC が音色を合成してスピーカーから出力する。本章では、機能一覧・システム構成図・操作インターフェース・状態遷移図を順に示す。なお、ファームウェアの設計には組み込み向けの構成手法 Embedded-Module-Architecture (以下 EMA) を全面的に採用する。

2.1 機能一覧

2.1.1 要求一覧

本システムが満たすべき要求を表 2.1 に示す。「(S)」は必達機能の完成後に着手するストレッチ要求である。R-Internal は外部から見えにくい設計上必要な内部要求を表す。

表 2.1 システムの要求一覧

ID	要求	内容
R-1	6 台同期演奏	指揮者 1 台と楽器 5 台の計 6 台が、指揮者の拍に合わせて同期して演奏する
R-2	輪唱曲の再生	主旋律 4 パート（金管）とリズム 1 パート（ドラム）からなる輪唱曲を演奏する。各パートは入り拍をずらして輪唱とする
R-3	可変テンポ追従	指揮者の振りの速さの変化に、楽器側のテンポが追従する
R-4 (S)	強弱制御	指揮者の振りの大きさに応じて発音の強弱（velocity）が変化する
R-Internal-1	通信の信頼性	数％程度のパケットロスが起きても演奏が破綻しない
R-Internal-2	CPU リソース	各ノードが IMU サンプリングや UDP 受信を取りこぼさない処理量に収める
R-Internal-3	起動時間	電源投入から演奏可能まで、実演のセットアップとして許容できる数秒に収まる

2.1.2 機能分解

第 1 章の図 1.1 で示した機能分解構造を、実装単位の機能群へさらに分解する。各機能群と所属ノードを次に示す。

F1 指揮情報抽出 (node_01)

IMU6 軸の取得、信号前処理（ローパスフィルタとノルム計算）、拍検出（振り下ろしのピーク検出）、テンポ推定（拍間隔から BPM を算出）、強弱推定（振幅から velocity を算出、ストレッチ）、演奏状態の管理。

F2 指揮情報配信 (node_01)

CTRL の送出（BPM・velocity・状態）、BEAT の送出（拍トリガ）、SoftAP の起動・維持（指揮者自身がアクセスポイントを兼ね、楽器ノードを収容する）。

F3 楽譜進行 (node_02～06)

CTRL/BEAT の受信、楽譜データの保持（C 配列・パート別）、BEAT 同期での音符進行、NOTE イベントの生成、パート固有ロジック（partId と入り拍の差し替え）。

F4 発音情報配信 (node_02～06)

NOTE パケットの生成、専用 PC への USB Serial 送信。

F5 音響合成 (PC)

NOTE の受信, 倍音構成と ADSR エンベロープによる音色合成, スピーカーへの出力。

F6 共通基盤 (全ノード)

IModule の登録と 3 フェーズループ, ModuleTimer による周期実行, SystemData / ProjectConfig への状態・設定の集約, 起動シーケンスと診断 LED 表示。

F1~F4・F6 はファームウェア (指揮者・楽器ノード) が, F5 は PC 側の Processing が担う。ストレッチ機能である F1 の強弱推定は, 必達機能の実装完了後に着手し, 初期実装ではインターフェースのみを用意してスタブで通す。分散配置した複数ノードを 1 つの拍源で同期させる構成は, ネットワーク演奏の先行研究 [4] にも見られる方式である。

2.2 システム構成図

2.2.1 全体構成

システムの全体構成を図 2.1 に示す。指揮者ノード node_01 は **SoftAP** (ESP32-S3 のソフトウェアアクセスポイント) を起動し, 楽器ノード node_02~06 はこれに STA として接続する。外部ルータやテザリングは不要である。node_01 は CTRL / BEAT を **UDP マルチキャスト** で全楽器ノードへ一斉配信する。各楽器ノードは楽譜を進行させながら発音情報 NOTE を生成し, **専用の PC へ USB Serial (1 対 1 接続)** で送信する。PC は NOTE を受け取って音色を合成し, スピーカーから出力する。通信は**ノード間 (指揮者→楽器) の WiFi UDP** と**楽器→PC の USB Serial** の二系統に分離している。

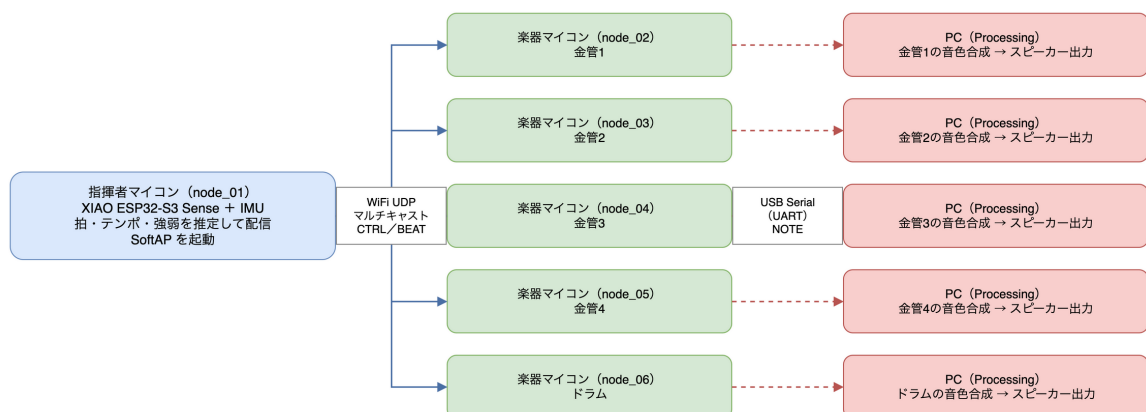


図 2.1 システム全体構成（指揮者 1 台＋楽器 5 台＋ PC5 台。実線＝ WiFi UDP マルチキャスト，破線＝楽器と PC の 1 対 1 USB Serial）

2.2.2 ノードとパートの対応

本システムは課題曲を 5 パートに分けて演奏する。課題曲の候補は、輪唱しやすくテンポ追従の確認がしやすい「かえるのうた」とする。各楽器ノードの partId・楽器・入り拍・役割を表 2.2 に示す。金管 4 パートは同じ主旋律を入り拍をずらして演奏することで輪唱を構成し、ドラムは 1 拍ごとのリズム伴奏を担う。

表 2.2 ノードとパートの対応

ノード	partId	楽器	入り拍	楽譜上の役割
node_02	0x02	金管 1	0 拍	主旋律を曲頭から演奏する
node_03	0x03	金管 2	4 拍	主旋律を 4 拍遅れて演奏する
node_04	0x04	金管 3	8 拍	主旋律を 8 拍遅れて演奏する
node_05	0x05	金管 4	12 拍	主旋律を 12 拍遅れて演奏する
node_06	0x06	ドラム	0 拍	1 拍ごとのリズム伴奏を演奏する

2.2.3 ノードの役割分担

各ノードと PC の責務を表 2.3 に示す。楽器ノード node_02～06 のファームウェアは共通ソースとパート設定の差し替えで構成し（ProjectConfig と楽譜データのみを差分とする）、5 台分の別コードは書き分けない。

表 2.3 ノードの役割分担

ノード	主な責務
指揮者 node_01	XIAO ESP32-S3 Sense。IMU の読み取りから拍・テンポ・強弱を推定し、CTRL / BEAT を送出する。自身が SoftAP を起動して楽器ノードを収容する
楽器 node_02～06	Arduino UNO R4 WiFi。BEAT に同期して楽譜を進行させ、NOTE を専用 PC へ送出する。金管 4 台とドラム 1 台で構成する
PC (Processing) 5 台	楽器ノードごとに 1 台を割り当てる。USB Serial で NOTE を受信し、倍音と ADSR で音色を合成してスピーカーへ出力する

2.2.4 演奏中のデータフロー

演奏中における 1 拍ぶんの情報の流れを次に示す。

1. IMU (I2C 接続) が node_01 へ加速度・角速度のサンプルを渡す。
2. node_01 内のロジック applyPattern() が、フィルタ・拍検出・テンポ推定を実行する。
3. 振り下ろしを検出すると、node_01 は BEAT (拍番号とマスタ時刻) を node_02～06 へ UDP マルチキャストで配信する。
4. 各楽器ノードは受信した BEAT の拍番号から楽譜位置を算出し、発音すべき音符を決定する。
5. 各楽器ノードは NOTE (音高・強弱・発音長) を専用 PC へ USB Serial で送信する。
6. 各 PC は NOTE から波形を合成し、スピーカーへ出力する。
7. 以上と並行して、node_01 は 20 Hz で CTRL (BPM・velocity・状態) を送信し続ける。

2.2.5 採用アーキテクチャ (EMA)

全ノードのファームウェアは、組込み向けの構成手法 EMA に準拠する¹⁾。本システムに適用した EMA の中核ルールを次に示す。バイト単位のパケット定義や具体的なコードは第 3 章で具体化する。

- ・ **IModule インターフェース**：全ハードウェアモジュールが、init / updateInput / updateOutput / deinit の 4 メソッドを実装する。
- ・ **3 フェーズ実行モデル**：loop() は「入力→ロジック→出力」の順で実行する。
- ・ **SystemData 集約**：モジュール間で共有する状態を 1 つの構造体に集約し、モジュール同士の直接呼び出しを禁止する。
- ・ **ProjectConfig 集約**：ピン配置・定数・閾値などノード固有の設定を 1 つのヘッダに集約する。
- ・ **ロジックの位置づけ**：拍検出・テンポ推定・楽譜進行のようなハードウェアに触れない判断ロジックは、モジュールではなくロジック関数 applyPattern() に書く。

2.3 操作インターフェース設計

操作インターフェースを、**ユーザーが触れる物理インターフェース**（指揮棒・LED）と、**ノード間・PC 間の通信インターフェース**（CTRL / BEAT / NOTE）の二層に分けて示す。

2.3.1 ユーザー操作

- ・ **指揮棒**：IMU を搭載した指揮者ノード node_01 を指揮棒として手に持ち、通常の指揮動作（振り下ろし）で操作する。専用のボタン等は持たない。
- ・ **起動順**：楽器ノードを先に起動して SoftAP の待機状態にし、その後に指揮者ノードを起動する。指揮者の SoftAP 起動で楽器ノードが接続され、指揮者が指揮を始めると BEAT が流れて演奏が始まる。
- ・ **キャリブレーション**：指揮者ノードは起動後の約 2 秒間、静止時の加速度を平均して基準値とする。使用者は起動後 2 秒間、腕を自然に下ろして待つだけでよい。

1) <https://github.com/takushio2525/Embedded-Module-Architecture>

2.3.2 LED 表示仕様

各ノードは内蔵 LED の点滅パターンで自身の状態を提示する（表 2.4）。

表 2.4 LED 表示パターン

ノード	状態	LED パターン	意味
node_01	Idle	低速点滅（1 Hz）	起動直後・SoftAP 起動前
node_01	Calibrating	中速点滅（2 Hz）	静止基準の学習中（2 秒）
node_01	Conducting	点灯	通常演奏中
node_01	Fallback	高速点滅（5 Hz）	IMU または SoftAP の異常
node_02～06	Idle	低速点滅（1 Hz）	起動直後・SoftAP 未接続
node_02～06	WaitStart	中速点滅（2 Hz）	SoftAP 接続済・最初の BEAT 待ち
node_02～06	Playing	点灯	通常演奏中

2.3.3 通信メッセージ

ノード間および PC との通信には、性質の異なる 3 種類のメッセージを使い分ける（表 2.5）。

表 2.5 3 種類の通信メッセージ

種別	送信元→宛先	経路・送信契機	役割
CTRL	node_01 →楽器 全台	WiFi UDP・定期 (20 Hz)	BPM・velocity・状態を継続的に同期 する
BEAT	node_01 →楽器 全台	WiFi UDP・拍検 出時	楽譜ポインタを前進させるトリガ
NOTE	楽器ノード→専用 PC	USB Serial・楽譜 イベント発火時	発音を PC へ依頼する

CTRL は 20 Hz で定期送信される冪等なメッセージであり、1 つ取りこぼしてもすぐ次が届くため、パケットロスの影響を受けにくい。BEAT は 1 拍に 1 回送る即時性重視のメッセージで、確実性を上げるため同じ拍番号を複数回連送し、受信側は拍番

号の重複を排除する。楽器ノードは BEAT が長く届かない間も演奏状態を保持し、次の BEAT 到来時に自然に再開する（指揮者の不在中に楽器だけが暴走しないよう、仮想的な拍を内挿して走り続ける方式は採らない）。NOTE は楽器ノードと PC を結ぶ 1 対 1 の USB Serial で送るため、ネットワーク輻輳やパケットロスの心配なく確実に到達する。バイト単位のパケット定義は第 3 章で示す。

2.4 状態遷移図

各ノードの演奏状態を有限状態機械として定義する。状態は SystemData の中に保持し、ロジック関数 applyPattern() が遷移を判定する。

2.4.1 指揮者ノード (node_01)

指揮者ノードの状態遷移を図 2.2 に、各状態の定義を表 2.6 に示す。

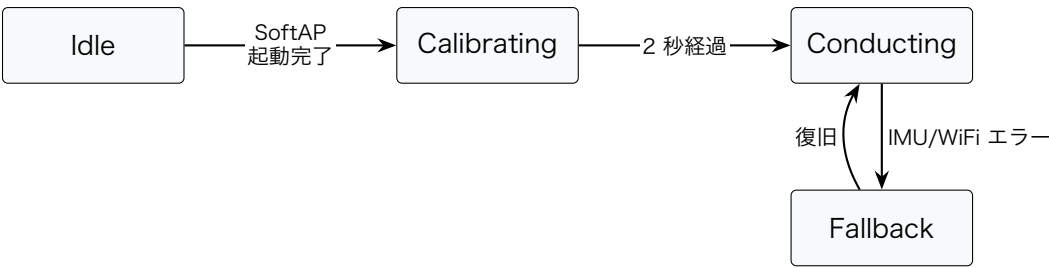


図 2.2 指揮者ノードの状態遷移

表 2.6 指揮者ノードの状態定義

状態	意味	動作
Idle	起動直後・SoftAP 起動前	IMU の読み取りは動くが拍検出は無効。CTRL / BEAT は送らない
Calibrating	初期静止状態の学習 中 (2 秒)	加速度ノルムの平均を静止基準として記録する
Conducting	通常演奏中	全モジュールが稼働。拍検出時に BEAT を、20 Hz 周期で CTRL を送出する
Fallback	エラー発生時	CTRL を状態 Fallback で送り続ける。BEAT は停止 し、LED を高速点滅する

2.4.2 楽器ノード (node_02~06)

楽器ノードの状態遷移を図 2.3 に、各状態の定義を表 2.7 に示す。楽器ノード 5 台はいずれも同じ状態機械で動作する。

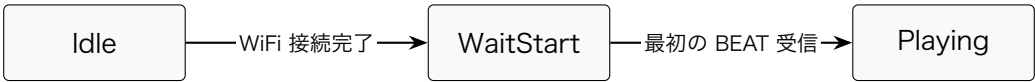


図 2.3 楽器ノードの状態遷移

表 2.7 楽器ノードの状態定義

状態	意味	動作
Idle	起動直後・SoftAP 未接続	受信モジュールは動くが楽譜進行は無効。SoftAP へ接続できるまでリトライする
WaitStart	SoftAP 接続済・最初の BEAT 待ち	CTRL / BEAT を受信し、指揮者との時計オフセットの学習を始める
Playing	通常演奏中	BEAT 受信ごとに拍番号から楽譜位置を算出して NOTE を送出する。BEAT が長く来なくても Playing に留まり、次の BEAT で自然に再開する

2.4.3 PC (Processing)

PC 側の Processing の状態遷移を表 2.8 に示す。Processing は起動後に USB Serial ポートを選んで接続し、NOTE を受信すると発音を始める。各 PC は 1 台の楽器ノードと 1 対 1 で接続するため、状態遷移はその 1 台との接続・受信のみを扱う簡潔な構成となる。

表 2.8 PC (Processing) の状態定義

状態	意味と主な処理
PortSelect Ready	Serial ポートの選択待ち。ポート一覧を表示し、担当者の選択を待つ 接続済み・NOTE 待ち。Serial バッファを空にし、NOTE の先頭同期を待つ
Playing	正常受信・発音中。NOTE を復号し、音色合成のための発音 (Voice) を生成する
Muted	受信は継続し発音のみ停止。新規の発音を作らず、既存の発音を減衰させる
Error	接続異常。Serial を閉じ、受信バッファを破棄して警告を表示する

第 3 章 システムの詳細設計

本章では、第 2 章の基本設計を、実装に着手できる水準まで具体化する。部品構成・ハードウェア配線・ソフトウェア構造・処理フロー・テスト設計を順に示す。記述の粒度は「実装方針が伝わること」を主眼とし、全関数・全設定値の網羅ではなく、設計判断の要点と代表値の提示にとどめる。コードのフル実装は本書の対象外とする。

3.1 部品一覧

第 1 章の図 1.2 で示した製品分解構造（PBS）を、実体の機材とソフトウェア要素の単位へ具体化する。ハードウェア構成を表 3.1 に、ソフトウェアの技術スタックを表 3.2 に示す。PC（Processing）とスピーカーは本書のスコープ外だが、NOTE パケットの受信境界として参考に併記する。

表 3.1 ハードウェア部品一覧

区分	要素	数量	備考
指揮者	XIAO ESP32-S3 Sense	1	ESP32-S3 (Xtensa LX7 デュアルコア)。内蔵 WiFi で SoftAP と UDP を直接制御する
指揮者	MPU6050 (GY-521)	1	6 軸 IMU。本案件は $\pm 4g$ / $\pm 2000\text{ dps}$ で使用, I2C アドレスは 0x68
指揮者	状態 LED	1	基板内蔵の LED_BUILTIN を流用する
楽器	Arduino UNO R4 WiFi	5	node_02~06 (金管 4 + ドラム)。同一ハードウェアで SoftAP に STA 接続する
楽器	状態 LED	5	基板内蔵の LED_BUILTIN を流用する
ネットワーク	node_01 の SoftAP で兼用	0	外部ルータ・テザリングは使わない
給電・通信	USB Type-C ケーブル	6	楽器 5 本は各 PC へ 1 対 1 直結 (給電と Serial を兼用), 指揮者 1 本は給電のみ
境界 (参考)	PC (Processing)	5	本書スコープ外。各楽器から NOTE を受信し音色を合成する
境界 (参考)	スピーカー	5	本書スコープ外。PC1 台あたり 1 系統

表 3.2 ソフトウェア技術スタック

レイヤ	採用技術	備考
MCU (指揮者)	XIAO ESP32-S3 Sense	ESP32-S3R8 (Xtensa LX7 240 MHz×2)
MCU (楽器)	Arduino UNO R4 WiFi	Renesas RA4M1 に WiFi 担当の ESP32-S3 を組み合わせた構成
フレームワーク・言語	Arduino framework / C++	PlatformIO 経由でビルドする
WiFi スタック	WiFi.h (指揮者) / WiFiS3 (楽器)	SoftAP と UDP マルチキャストを使う
設計パターン 共通層	EMA (ADR-0005) ModuleCore・OrcProtocol・OrcNetModule ほか	全ノードのファームウェアに適用する firmware/common/lib に集約し全ノードで共有する
PC 側	Processing 4	NOTE の受信と音色合成 (本書スコープ外)

3.2 ハードウェア設計

3.2.1 配線・ピンアサイン

指揮者ノード node_01 (XIAO ESP32-S3 Sense) は外付けの MPU6050 を I2C で接続する。XIAO の I2C 既定ピンは SDA=D4 (GPIO5), SCL=D5 (GPIO6) である。3.3 V / GND を MPU6050 へ給電し, AD0 を GND に落として I2C アドレスを 0x68 に固定する。楽器ノード node_02~06 (UNO R4 WiFi) は内蔵 LED 以外に外部配線を要しない。各ノードの配線を表 3.3 に示す。

表 3.3 配線・ピンアサイン

ノード	用途	ピン・接続
node_01	I2C SDA (MPU6050)	D4 (GPIO5)
node_01	I2C SCL (MPU6050)	D5 (GPIO6)
node_01	MPU6050 の電源	3.3 V・GND。AD0 を GND に落とし I2C アドレスを 0x68 に固定する
node_01	状態 LED	LED_BUILTIN (XIAO のユーザー LED)
node_01	給電	USB Type-C (給電のみ。開発・試験時のみデバッグ Serial を読む)
node_02~06	状態 LED	LED_BUILTIN
node_02~06	給電 + NOTE 送信	USB Type-C (各楽器が専用 PC へ 1 対 1 直結。Serial.write で NOTE を出力する)

3.2.2 ネットワーク構成

通信は二系統に分離する。**Arduino 同士**は WiFi UDP (node_01 が立てる SoftAP 配下) で, **Arduino と PC** は USB Serial の直結で通信する。node_01 は `WiFi.softAP("OrchestraAP", "orchestra2026")` でアクセスポイントを起動し, node_02~06 は `WiFiS3` で同じ SSID へ STA として接続する。SoftAP の既定アドレスは 192.168.4.1, 楽器側は内蔵 DHCP で 192.168.4.2 以降を取得する。node_01 は CTRL / BEAT を **UDP マルチキャスト** (239.0.0.1:5001) で全楽器ノードへ一斉

配信し、各楽器ノードは発音情報 NOTE を**専用 PC のシリアルポート**へ書き込む。接続トポロジは第 2 章の図 2.1 に示したとおりである。

指揮者ノードだけを XIAO ESP32-S3 としたのは、ESP32-S3 を直接制御すると WiFi.softAP() と UDP マルチキャストがネイティブにサポートされ、内蔵 WiFi を別チップ越しに扱う UNO R4 WiFi よりも SoftAP の制御自由度が高いためである。外部ルータやテザリングが不要なため、会場 LAN のセキュリティ制限や電波混雑の影響を受けない利点もある。

なお、WiFi の省電力モードが有効な STA が接続していると、マルチキャストフレームの配送がビーコン周期ぶん遅れる。これを避けるため楽器ノードの WiFi 省電力モードは無効化して運用する。無効化できない場合の代替策は、テスト設計 (3.5 節) の未確定事項として扱う。

3.3 ソフトウェア設計

3.3.1 ファイル構成

firmware/配下は、全ノードで共有する**共通層**と、**ノード別プロジェクト**の 2 段構造とする。EMA に準拠し、PlatformIO の lib_extra_dirs 設定で共通層を全ノードから参照する。ディレクトリ構成の抜粋を次に示す。

```
firmware/
|-- common/lib/           # 全ノード共通 (lib_extra_dirs で参照)
|   |-- ModuleCore/      #   IModule + ModuleTimer
|   |-- OrcProtocol/     #   CTRL/BEAT/NOTE パケット定義
|   |-- OrcNetModule/    #   WiFi 接続維持 + UDP 送受信
|   |-- StatusLedModule/ #   状態 LED 出力
|   |-- SerialDebug/     #   SERIAL_DEBUG 切替のデバッグ出力
|-- node_01/             # 指揮者ノード
|   |-- include/         #   SystemData.h, ProjectConfig.h
|   |-- src/             #   main.cpp, applyPattern.cpp
|   |-- lib/             #   ImuModule, OrcSenderModule
|-- node_02 ~ node_06/   # 楽器ノード (金管 4 + ドラム。5 台とも同構造)
|   |-- include/         #   SystemData.h, ProjectConfig.h, score_data.h
|   |-- src/             #   main.cpp, applyPattern.cpp, score_data.cpp
|   |-- lib/             #   OrcReceiverModule, NoteSenderModule
```


楽器ノード 5 台は同一コードで動作し、ノード間の差分は ProjectConfig.h (partId・入り拍) と score_data.* (楽譜配列) にのみ閉じ込める。ビルド時の SERIAL_DEBUG フラグで、本番動作 (NOTE のバイナリ送信) と開発時 (テキストログ出力) を切り替える。

3.3.2 オブジェクト設計 (クラス図)

すべてのハードウェアモジュールは、EMA の抽象基底 IModule (init / updateInput / updateOutput / deinit の 4 メソッド) を継承する。WiFi UDP を扱う OrcNetModule は共通層に置き、指揮者・楽器の両ノード種で共有する (図 3.1)。各モジュールの責務を表 3.4 に示す。

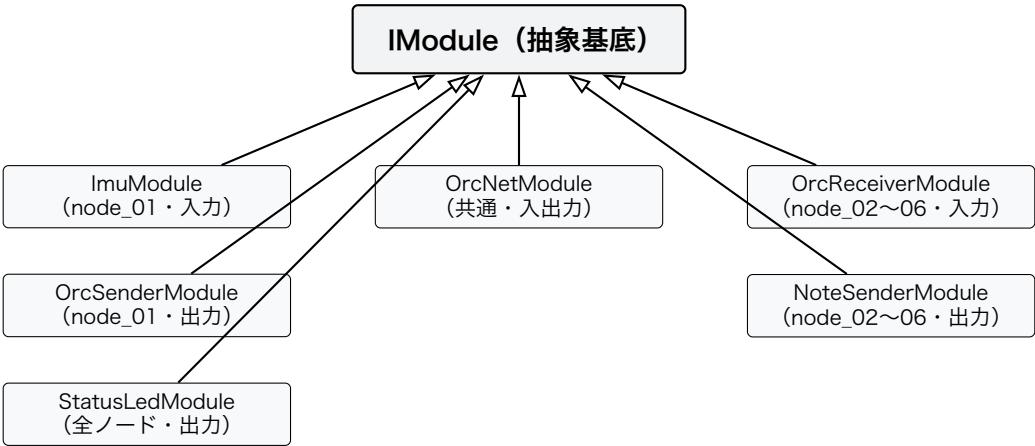


図 3.1 モジュールのクラス継承関係

表 3.4 モジュール一覧と責務

モジュール	ノード	分類	責務
ImuModule	node_01	入力	MPU6050 (I2C) の 6 軸を読み取る
OrcSenderModule	node_01	出力	CTRL / BEAT の送信内容を組み立てる
OrcReceiverModule	node_02～06	入力	受信した CTRL / BEAT を整形する
NoteSenderModule	node_02～06	出力	NOTE を USB Serial で PC へ送信する
OrcNetModule	全ノード	入出力	WiFi 接続の維持と UDP の送受信
StatusLedModule	全ノード	出力	状態に応じた LED の点滅表示

PC 側の Processing は、シリアル入力をフレーム同期して 20 バイトの NOTE を切り出す受信部、Serial スレッドと描画スレッドを分離する受信キュー、発音単位 (Voice) の生成と消音を管理する発音管理部、および操作画面の描画部から成る。詳細は PC 側担当 (梅澤) の設計に委ねる。

3.3.3 通信パケット設計

ノード間および PC との通信に用いる 3 種類のパケット (CTRL / BEAT / NOTE) は、いずれも **20 バイト固定長**とする。先頭 12 バイトが全パケット共通のヘッダ (表 3.5)、後続 8 バイトが型ごとのペイロード (表 3.6) である。多バイト値はリトルエンディアンで配置する。先頭の magic は WiFi UDP 上ではプロトコル識別子、USB Serial 上では**フレーム同期マーク**を兼ねる。PC 側はバイト列を走査し、magic が現れた位置をフレーム先頭とみなして固定 20 バイトを 1 パケットとして読む。

表 3.5 通信パケットの共通ヘッダ (先頭 12 バイト)

オフセット	サイズ	フィールド	内容
0	2 B	magic	固定値 0x4F52 (ASCII "OR")。プロトコル識別子兼フレーム同期マーク
2	1 B	version	プロトコルバージョン (現行 0x01)
3	1 B	type	パケット種別 (1=CTRL, 2=BEAT, 3=NOTE)
4	4 B	seq	送信側で単調増加させる連番。パケットロスの検出に使える
8	4 B	times-tampMs	送信時のマスタ時刻 (指揮者の millis())。楽器側の時計同期に使う

表 3.6 型別ペイロード（共通ヘッダに続く 8 バイト）

オフセット	サイズ	フィールド	内容
CTRL (type=1) : 指揮者→楽器, 20Hz で定期配信			
12	2 B	bpmQ8	BPM を 8 倍した整数（分解能 0.125 BPM。例：120.5 BPM → 964）
14	1 B	velocity	強弱（0-127）。ストレッチ未実装時は固定値 64
15	1 B	state	指揮者の状態（0=Idle, 1=Calibrating, 2=Conducting, 3=Fallback）
16	4 B	予約	ゼロ埋め（将来拡張用）
BEAT (type=2) : 指揮者→楽器, 拍検出時に 2 連送			
12	2 B	beatNo	曲頭からの拍番号（巻き戻さず単調増加させる）
14	2 B	予約	ゼロ埋め
16	4 B	playAtMasterMs	この拍をマスタ時刻の何 ms に発音するか指定
NOTE (type=3) : 楽器→PC, 楽譜イベント発火時に USB Serial で送信			
12	1 B	partId	発音元パート（0x02~0x06。金管 0x02~0x05, ドラム 0x06）
13	1 B	noteNumber	MIDI ノート番号（60=C4）。ドラムは GM ドラムマップとして解釈する
14	1 B	velocity	強弱（0-127）。楽譜値と CTRL 値を合成した後の値
15	1 B	gate	1=NoteOn, 0=NoteOff
16	2 B	durationMs	発音予定の長さ。PC 側の自動消音に使う
18	2 B	予約	ゼロ埋め

3.3.4 楽譜データ形式

楽譜は ScoreEvent 構造体の配列 kScore[] としてプログラムメモリに埋め込み、SD カード等の外部ストレージは使わない。1 イベントが 1 拍に対応し、曲頭から休符込みで時系列順に並べる。各フィールドを表 3.7 に示す。テンポは楽譜に持たせず、CTRL で届く BPM から実時間へ変換する。

表 3.7 ScoreEvent (楽譜 1 イベント) のフィールド

フィールド	型	意味
beatAt	uint16	何拍目かを示す参考値。発火条件には使わず、楽譜を読みやすくするための冗長情報
noteNumber	uint8	MIDI ノート番号。0 は休符
velocity	uint8	楽譜上の強さ (0-127)。CTRL の velocity と乗算合成する
durationQ8	uint16	発音長。1 拍の 1/256 単位で表す (256=1 拍, 128=8 分音符)
flags	uint8	bit0=NoteOn / bit1=NoteOff (拡張用) / bit2=休符
sub 系	uint8 / uint16	細分音符 (拍頭からずれて鳴る 2 音目) の音高・強さ・オフセット・長さ。8 分音符などの表現に使う

発音長は、CTRL で届く BPM から $\text{durationMs} = (60000/\text{BPM}) \times (\text{durationQ8}/256)$ で実時間に換算する。楽譜は拍単位で記録するため、テンポが変わっても配列を書き換える必要はない。

楽譜上の発火位置は、受信した BEAT の beatNo から $\text{idx} = (\text{beatNo} - \text{startBeatNo}) \bmod \text{kScoreLength}$ で算出する (startBeatNo はそのパートの入り拍)。剰余演算により曲末から曲頭へ自動でループ再生される。各楽器ノードの partId と入り拍は第 2 章の表 2.2 に示したとおりで、ファームウェアはこれらと楽譜配列だけを差し替える。NoteOff パケットは送らず、PC 側が durationMs を読み取って自動消音する。これは、Arduino 側から NoteOff を送る方式では USB CDC のバッファリングにより「音が鳴りっぱなし」になりやすいためである。

3.3.5 時刻同期方式

WiFi UDP マルチキャストの到達時刻には数 ms~十数 ms のばらつき (ジッタ) が乗る。これをそのまま発音に使うと楽器間の同期が運任せになる。本設計では、このジッタを**時計同期の誤差に押し付ける**ことで、楽器間の相対的な発音タイミングを小さく抑える。

中核となる考え方は、指揮者ノード (SoftAP) の millis() を**マスタ時刻**として全ノードで共有することである。BEAT は「マスタ時刻 playAtMasterMs に発音せよ」という**未来時刻の指定**として送る。楽器は自分の時計をマスタ時刻に変換するオフセッ

トを学習し、変換後の時刻が指定時刻に達した瞬間に発音する。WiFi の到達時刻がばらついても、全楽器が共通のマスタ時刻に揃うため発音タイミングは合致する。

オフセットの推定には、共通ヘッダの `timestampMs` を用いる。楽器は CTRL / BEAT を受信するたびに「送信時のマスタ時刻と自時計の差」を 1 サンプルとして取り出し、係数 0.10 の指数移動平均 (EMA) で平滑化する。CTRL が 20Hz で常時流れるため、数発でオフセットは収束する。

指揮者は BEAT 送出時に `playAtMasterMs = masterNow + beatLookaheadMs` (仮値 50ms) を載せる。楽器は、指定時刻が未来なら到達まで待機し、受信が遅れて指定時刻を過ぎていても**捨てずに即発音する**。指定時刻を過ぎたものを捨てる方式は、通信のリトライや起動直後に「音が鳴らない」原因になりやすいためである。代わりに BEAT を 2 連送し、どれか 1 発が指定時刻前に届けば発音が揃う構成とする。`beatLookaheadMs` は振り下ろしから発音までの固定遅延としてそのまま乗るが、映像に対し音が遅れる方向のずれは許容されやすく、本用途では受け入れる。

3.3.6 音色合成 (PC 側)

PC 側の Processing は、NOTE の `noteNumber` を周波数へ変換し、倍音を重ねた波形に ADSR エンベロープを掛けて音色を合成する。金管音色の合成パラメータの初期値を表 3.8 に示す。音色定義は JSON ファイルとして外部化し、楽器の追加・調整をファイル追加だけで行える構成とする。本節は PC 側担当 (梅澤) の設計範囲であり、要点のみを示す。

表 3.8 金管音色の合成パラメータ (初期値)

要素	初期値と方針
倍音構成	基音から第 7 倍音までを重ねる。振幅比は 1.00 / 0.70 / 0.50 / 0.30 / 0.20 / 0.10 / 0.05
ADSR エンベロープ	Attack 20ms / Decay 40ms / Sustain 0.75 / Release 60ms。 Sustain は velocity でスケールする
ドラム音色	金管とは別に短い減衰音として合成し、 <code>noteNumber</code> で Kick / Snare / Hi-hat 風に切り替える

3.4 処理フローの設計

3.4.1 3 フェーズ実行モデル

全ノード共通で、`loop()` は「入力→ロジック→出力」の 3 フェーズを毎周期この順で実行する（表 3.9）。入力モジュールは `SystemData` へ書き込み、出力モジュールはこれを読み取る。モジュール同士の直接呼び出しは禁止し、結合は `SystemData` の一点に閉じる。`OrcNetModule` は入力・出力の両配列に登録し、「入力フェーズで先に受信し、出力フェーズの最後で送信する」よう配列順で制御する。

表 3.9 `loop()` の 3 フェーズ実行順序

フェーズ	処理内容
1. 入力	入力モジュールを順に <code>updateInput()</code> で呼ぶ。センサ値や UDP 受信結果が <code>SystemData</code> に書き込まれる
2. ロジック	<code>applyPattern()</code> を 1 度呼ぶ。フィルタ・拍検出・状態遷移・楽譜進行はすべてここに集約する
3. 出力	出力モジュールを順に <code>updateOutput()</code> で呼ぶ。LED 表示・UDP 送信・Serial 送出が反映される

3.4.2 指揮者ノードの処理フロー

指揮者ノードのロジック `applyPattern()` は、毎周期おおむね次の順で処理する。

1. **信号前処理**: IMU の加速度を一次ローパスフィルタ（係数 0.10）で平滑化し、ノルムを計算する。キャリブレーション中に採った静止時ノルムを差し引き、振りの強さを表す動加速度を得る。
2. **拍検出**: 動加速度から振り下ろしを検出する（次段落）。
3. **テンポ推定**: 拍間隔から瞬時 BPM を求め、EMA（係数 0.30）で平滑化する。40～240 BPM の範囲にクランプする。
4. **状態遷移**: `Idle` → `Calibrating` → `Conducting` と進み、IMU または WiFi の異常で `Fallback` へ落ちる。

5. **出力反映**：状態に応じて LED の点滅を設定し、CTRL / BEAT の送信内容を組み立てる。

拍検出では、単純なピーク閾値判定だと「ピーク後の発火で振りと音がずれる」「振り戻しを 2 拍目に拾う」「軽い揺れで誤発火する」といった問題が出る。これを構造的に避けるため、**ゲート式ステートマシン** (Idle / Armed の 2 状態) を用いる。動加速度ノルムが閾値 (1.20g) を超えると Armed に入り、Armed 中だけ動加速度を時間積分して擬似的な振りの経路長を求める。経路長が 0.20m に達した瞬間に拍を発火するため、ピークを待たず早期に発火でき、振りと音のずれが小さい。発火後は 350ms の不応期を設け、振り戻しの誤検出を防ぐ。判定をスカラーの動加速度ノルムで行うため、振る向き (縦・横・斜め) にほとんど依存しない。

出力では、拍を検出した周期に BEAT を送信予約し、playAtMasterMs に発音指定時刻を載せる。CTRL は 20 Hz の周期で常時送信予約する。実際の UDP 送信は、出力フェーズ後段の OrcNetModule が担当する。

3.4.3 楽器ノードの処理フロー

楽器ノードは、受信した BEAT ごとに「時計同期→発音判定→楽譜進行→NOTE 送出」を行う (図 3.2)。ロジック applyPattern() の処理は次のとおりである。

1. **時計同期**：受信した CTRL / BEAT の timestampMs と自時計の差を、EMA (係数 0.10) で平滑化し、マスタ時刻へのオフセットを更新し続ける。
2. **状態遷移**：WiFi 接続で WaitStart へ、最初の BEAT 受信で Playing へ遷移する。
3. **発音判定**：保留中の BEAT について、発音指定時刻 playAtMasterMs をマスタ時刻が過ぎたら発火する。受信が遅れて指定時刻を過ぎていても捨てずに即発火する。
4. **楽譜進行**：発火した BEAT の beatNo から楽譜位置を算出する。入り拍に達していなければ何も鳴らさない (輪唱の入り待ち)。BEAT を取りこぼしても、次の BEAT の beatNo から算出される位置が正しいため自己修復する。
5. **音符生成**：その位置が休符でなければ、楽譜 velocity と CTRL velocity を乗算合成し、NOTE の発音情報を組み立てる。
6. **出力反映**：状態に応じて LED を更新する。NOTE は出力フェーズで NoteSenderModule が 20 バイトのバイナリとして Serial へ送出する。

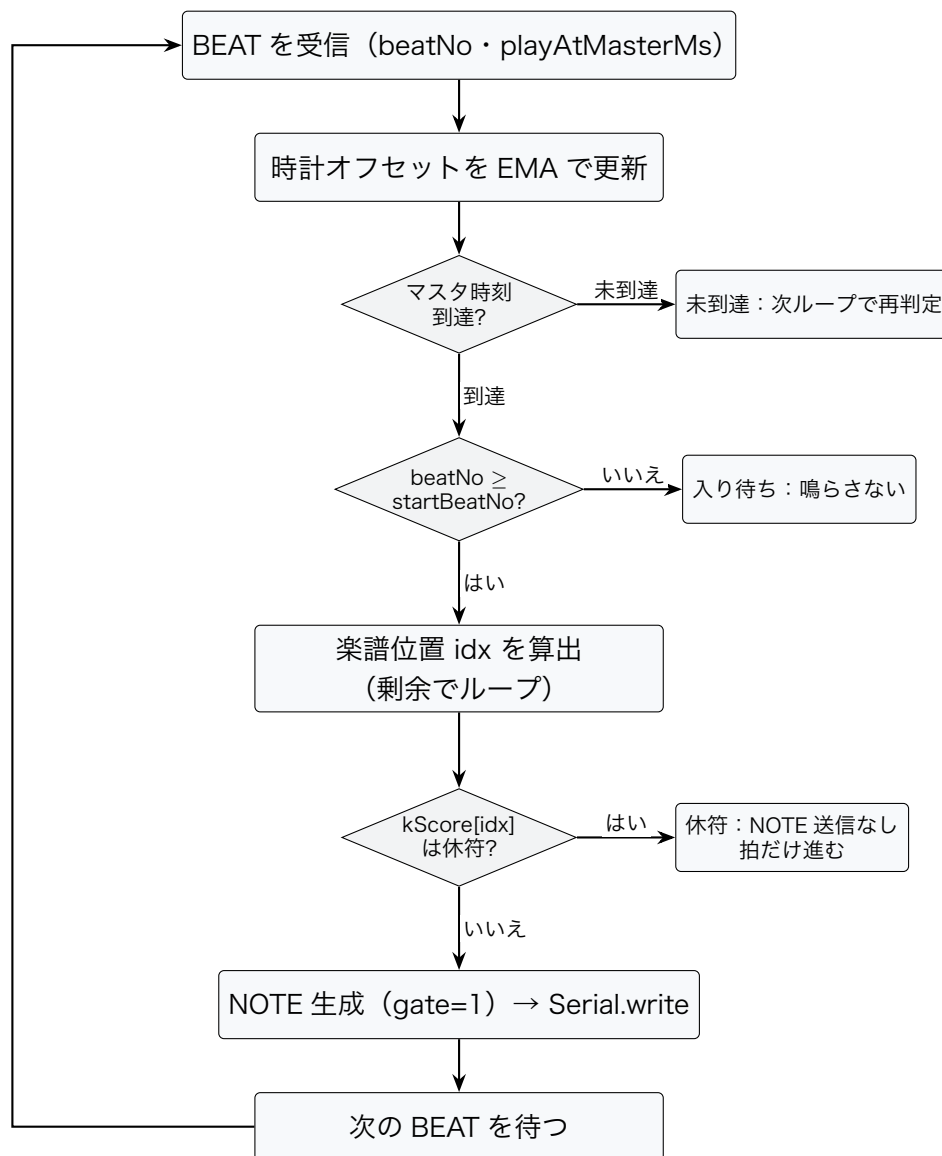


図 3.2 楽器ノードの楽譜進行と NOTE 送信の処理フロー

3.5 テストの設計

本節では、設計が満たすべき数値目標と、それを段階的に確認する検証の方向性、および実機検証で詰める仮値を示す。具体的な試験ケースや計測手順は実装フェーズで確定する。

3.5.1 設計目標値

本システムの設計が狙う数値目標を表 3.10 に示す。これらは「テストの合否基準」ではなく、設計判断の前提として置いた値であり、実機計測で再調整する想定である。

表 3.10 設計目標値

項目	目標値	根拠・位置づけ
楽器間の同期誤差	$\leq 20 \text{ ms}$	ADR-0006 でチーム合意した有効性指標。数十 ms 程度のずれは聴き手にほぼ気づかれないという知見から安全側に設定した
拍検出の正解率	$\geq 90 \%$	定速指揮時に演奏が破綻しない実用水準として置いた
テンポ追従の遅延	$\leq 2 \text{ 拍}$	人間の演奏者が BPM 変化に追従するのと同等を最低水準とした
入力フェーズの CPU 時間	$\leq 2 \text{ ms}$	IMU を 200 Hz (周期 5 ms) で読むため、3 フェーズが各 1.7 ms 程度に収まる必要がある
起動時間	$\leq 5 \text{ s}$	WiFi 接続・DHCP に 2~3 秒、キャリブレーションに 2 秒を見込む
強弱制御（ストレッチ）	3 段階以上	p / mf / f の 3 段階で初級~中級曲のダイナミクスを再現する

3.5.2 検証の方向性

設計の妥当性は、実装後に「単体→結合→実機」の 3 段階で確認していく方針とする。

- ・ **単体**：判断ロジックを IModule 派生ではなく applyPattern() 関数に書くため、SystemData をテスト側で組み立てて関数を呼ぶだけで、拍検出・テンポ推定・楽譜進行・時計同期のロジックを切り出して確認できる。パケットのシリアルイズもバイト列のラウンドトリップで確認する。
- ・ **結合**：指揮者ノード単体→指揮者+楽器 1 台→指揮者+楽器 5 台、と接続規模を段階的に広げ、BEAT 受信から NOTE 出力までの経路と、5 台の発音タイミング差を確認する。

- ・ **実機**：全ノードと PC で実音を再生し、指揮の速度・振幅を変えたときのテンポ追従と強弱の挙動を、聴感と計測の両面で確認する。BEAT を意図的に一部破棄した条件での演奏継続もこの段階で確認する。PC 側の Processing は、ビルド確認・NOTE 復号・自動消音・同時発音数の上限などを個別に確認する。

3.5.3 実機検証で確定する仮値

設計値のうち、実機での段階的な結合確認で詰めるもの、および楽譜・音色の仕様確定を待って実装するものを表 3.11 に示す。

表 3.11 実機検証で確定する主な仮値・未確定事項

項目	現状と確定の見極め
BEAT の連送回数	仮値 2 連送。結合確認でパケットロスと同期誤差を見て 1～3 連送で調整する
lookahead 時間	仮値 50ms。振り→音の体感遅延と、遠い楽器の到達率を見て 30～80ms で調整する
時計同期の EMA 係数	仮値 0.10。オフセットの収束時間と定常ジッタを見て 0.05～0.20 で調整する
片道遅延の系統差	5 台でほぼ同程度と仮定し補正なし。系統差が無視できなければ往復測定で各台を較正する
WiFi の STA 省電力モード	無効化して運用する。無効化できない場合は lookahead を拡大して配送遅延を吸収する
WiFi チャンネル	仮値 6。会場の電波状況を調査して空きチャンネルへ変更する
強弱推定（ストレッチ）	現状は velocity 固定 64 のスタブ。必達機能の完成後に振り幅→velocity の写像を実装する
ドラムパート (node_06)	金管と同じ ScoreEvent で実装する。noteNumber は GM ドラムマップに固定する

参考文献

- [1] 川合雄佑, 伊藤彰教, 「非接触型電子楽器の開発を通じたサウンドデザイン」, 東京工科大学メディア学部, 先端芸術音楽創作学会会報, Vol. 14, No. 1, pp. 14–19, 2022.
- [2] 任天堂株式会社, 「Wii Music 公式サイト」, <https://www.nintendo.co.jp/wii/r64j/index.html>, 2008 (2026-04-28 閲覧) .

- [3] Álvaro Barbosa, “Displaced Soundscapes: A Survey of Network Systems for Music and Sonic Art Creation,” *Leonardo Music Journal*, Vol. 13, pp. 53–59, 2003.
- [4] 久松剛, 「分散環境におけるフィードバックを用いたオーケストラ演奏機構の構築」, 慶應義塾大学 環境情報学部 卒業論文, 2003.
- [5] 中村栄太, 武田晴登, 山本龍一, 齋藤康之, 酒向慎司, 嵯峨山茂樹, 「任意箇所への弾き直し・弾き飛ばしを含む演奏に追従可能な楽譜追跡と自動伴奏」, 情報処理学会論文誌, Vol. 54, No. 4, pp. 1338–1349, 2013.
- [6] 竹内英世, 梅崎太造, 保黒政大, 「カラオケ採点用の高分解能ピッチ抽出法」, 電気学会論文誌 C, Vol. 129, No. 10, pp. 1889–1901, 2009.

第 4 章 生成 AI の利用